

1. Agenda

Sprecher: Der Schwerpunkt liegt bewusst auf dem Setup, weil das Fundament zuerst stehen muss.

2. Forschungsfrage

- Pipeline: **STT** → **LLM** → **TTS** (sequenziell)
- Latenz entscheidet die Nutzbarkeit
- **Kernfrage:** Netz/Infrastruktur vs. Backend?
- Faktoren: **RTT, Protokoll, Hosting-Region**
- Bewusst offen — **auch ein Nein zählt**
- Zweite Achse: **Verfügbarkeit** (separat)

Sprecher: Ich frage, in welchem Maße Netznähe gegenüber der Backend-Verarbeitung die Latenzunterschiede erklärt — und 'Netz erklärt weniger' ist eine gültige Antwort.

1 · SETUP Setup

- Vantage Point
- Provider-Matrix
- Messdesign

Sprecher: Das Setup ist der wichtigste Teil — hier zeige ich, womit und wie sauber gemessen wird.

3. Vantage Point

- **AWS EC2 eu-central-1** (Frankfurt)
- **c6i.large** — bewusst NICHT burstable
- Burstable würde drosseln → **Latenz-Sprünge ohne Netz-Ursache**
- **CPU-Steal-Time** je Slot geloggt
- **IPv4** durchgängig erzwungen
- **OpenSSL 3.x** (LibreSSL meldete TLS-Zeiten falsch)
- Limitation: **ein Vantage Point** — Kernbefund unberührt

Sprecher: Ich nutze bewusst eine nicht-burstable Instanz und logge CPU-Steal-Time, damit kein Throttling als Latenz fehlinterpretiert wird.

4. Provider-Matrix · 9 Endpunkte

- **9 Endpunkte** über 6 Anbieter × 3 Kategorien
- Feste Inputs je Kategorie
- Fairer Cross-Provider-Vergleich

Sprecher: Dieselben neun Endpunkte über drei Kategorien, mit fixen Inputs je Kategorie für einen fairen Vergleich; die Anbieter-Auswahl ist in der Methodik begründet.

5. Messdesign · Cold-Start · interleaved

- **Cold-Start:** frische TCP+TLS je Call
- Kein Pooling · kein Keep-Alive
- **Feste Inputs** je Kategorie
- STT 5 s-WAV · LLM Capital-Prompt · TTS Gruß
- **Interleaved** Round-Robin, 100 Runden
- Startreihenfolge rotiert
- Fehlschlag blockiert die Runde nicht

Sprecher: Jeder Call baut eine frische Verbindung auf — gemessen wird der Cold-Start; der LLM-Input ist exakt 'Reply in one short sentence: What is the capital of Germany?' (max_tokens=50, streaming).

2 · METHODIK Methodik

- 3 Schichten · RTT · Edge/Host
- Paket-Eichung · Metrik-Uhr
- Kampagne & Aggregation

Sprecher: Jetzt zeige ich, wie ich messe und warum die Zahlen valide sind — inklusive der beiden kritischen Punkte zu Edge-RTT und Timer-Vertrauen.

6. Drei-Schichten-Architektur

- Die Frage liefert die **Zerlegung**
- **Layer 1** → **Host** · **Layer 3** → **volle URL**
- Differenz = **Backend (Hardware + Modell)**
- **Layer 2** macht die Leitung prüfbar

Sprecher: Layer 1 misst zum Host, Layer 3 über die volle URL — die Differenz ist der Backend-Anteil, und Layer 2 macht die Leitung paketweise überprüfbar.

7. Layer 1 · RTT via TCP-Handshake

- **TCP-Ping** (SYN→SYN-ACK :443) = primär
- Vergleichbar bei **allen 9** (ICMP teils geblockt)
- Trifft den **echten API-Port**
- Deckt sich mit dem **Connect-Timer aus Layer 3**
- Validierung: **TCP-Ping** ≈ **ICMP-Ping**

Sprecher: TCP-SYN→SYN-ACK auf Port 443 ist eine RTT, funktioniert bei allen neun und entspricht genau dem Connect-Timer aus Layer 3; wo ICMP geht, validiere ich gegen den ICMP-Ping.

8. Layer 1 · Edge oder Host?

- **Erklärung:** Cloudflare-Edge FRA, nicht Backend
- ~1 ms = reale Edge-Eigenschaft, kein Fehler
- **Edge nur, wenn alle drei zutreffen:**
- (a) **einstellige ms** aus FRA
- (b) Ziel-IP in **AS13335** (CDN)
- (c) Traceroute endet **im CDN-AS**

Sprecher: Die niedrigen RTTs sind eine reale Cloudflare-Edge-Eigenschaft in Frankfurt — belegt über drei Bedingungen — und gelten nur für 4 von 9 Endpunkten; niedrige RTT heißt nicht automatisch Edge, Azure mit ~11 ms ist ein echter Host.

9. Layer 2 · Paket-Eichung des Connect-Timers

- **30 Cold-Starts je Provider** (paketvalidiert: 30/30 bzw. 28/30)
- App-Timer vs. **SYN→SYN-ACK** (PCAP)
- Geeicht: **nur der Connect-Timer**
- ttft/ttfa: gleiche Uhr, aber **nicht direkt** paket-geeicht
- Layer 2 = **Konsistenz-Check**, kein 2. Beweis

Sprecher: Der Connect-Timer ist auf ~0,1 ms paketgenau geeicht — aber ehrlich nur er; ttft und ttfa nutzen dieselbe Uhr, starten erst beim Request-Absenden und sind nicht direkt paket-geeicht, deshalb ist Layer 2 ein Konsistenz-Check, kein zweiter Beweis.

10. Metrik-Asymmetrie · ab wann läuft die Uhr?

- **Klar deklariert:** ab wann die Uhr läuft
- Jeder Call ist Cold-Start: Connect **immer** gemessen
- STT **ttfp**: nur Engine ab erstem Audio-Chunk (Connect **separat**)
- LLM/TTS **ttft/ttfa**: Connect **enthalten** (ab Request)
- E2E: jeder Anbieter-Connect **genau einmal**

Sprecher: Jeder Call ist ein Cold-Start mit frischer Verbindung — auch STT; bei STT wird der Connect aber als eigene Zahl geführt, und ttfp misst nur die Engine ab dem ersten Audio-Chunk (fairer Vergleich, weil die STT-Anbieter unterschiedlich weit weg sind: Azure ~11 ms, Deepgram/Rev.ai ~140 ms). LLM/TTS messen ab dem Request-Absenden, der Connect ist dort enthalten. In der E2E-Summe zählt jeder der drei Anbieter-Connects genau einmal: den STT-Connect addiere ich explizit, LLM- und TTS-Connect stecken schon in ttft/ttfa — so wird keiner doppelt gezählt; und es geht das STT-Final ein, nicht ttfp.

11. Kampagne & Aggregation

Sprecher: 56 Slots = 50.400 Calls, alle erhoben und A4-ausgewertet (Vollkampagne); Bootstrap-CI noch ausstehend; die Headline ist der robuste Median der Slot-Mediane.

3 · ANALYSE Analyse

- Kernbefund
- Interpretation
- Limitationen

Sprecher: Alle folgenden Zahlen sind die A4-Auswertung über alle 56 Slots (Vollkampagne); Bootstrap-CI noch ausstehend — die Aussage selbst ist davon unberührt.

12. Kernbefund · gleicher Edge, ~7,3×

- Alle 3 LLM @ **Cloudflare FRA**
- **100 % des Traffics** belegt (2 CF-IPs, AS13335, ~1 ms)
- ttft streut **~7,3×** (Groq→Mistral→OpenAI)
- **Pro IP stabil** (kein Edge-Effekt)
- **Region-Reihenfolge gedreht:** Mistral (EU) langsamer als Groq (US)
- Gleiches Netz, ~7,3× → **Netznähe erklärt das nicht**

Sprecher: Alle drei LLM terminieren am selben Cloudflare-Edge in Frankfurt mit ~1 ms — für 100% des Traffics belegt — und trotzdem streut ttft etwa 7,3-fach und ist pro IP stabil; die deklarierte Region ist sogar gedreht. Dasselbe Netz kann die Spreizung nicht erklären.

13. Interpretation · der Kompass

- Schritt zu Backend trägt **Modellgrößen-Confound**
- Groq = kleinstes Modell + LPU
- → **Backend (Hardware + Modell) statt Geografie**
- **2. Beleg (TTS):** OpenAI-TTS @ CF-FRA → ~942 ms ttfa = fast **reines Backend** (connect ~1 ms)

Sprecher: Verteidigbar ist nur die negative Aussage; der Schritt zu 'Backend' trägt den Modellgrößen-Confound — Groq ist das kleinste Modell auf LPU-Hardware — deshalb 'Backend (Hardware + Modell) statt Geografie'; OpenAI-TTS am selben Edge bei ~942 ms ttfa (connect ~1 ms → fast reines Backend) ist ein zweiter Beleg am gleichen Edge.

4 · ABSCHLUSS Abschluss

- Stand
- Nächste Schritte
- Diskussion

Sprecher: Zum Abschluss der Stand, die nächsten Schritte und eine Frage an Sie.

14. Stand & nächste Schritte

- **Messinfrastruktur steht** und läuft stabil
- → offene Punkte betreffen nur **Doku/Reporting**, nicht die Daten
- Stop ~23.06. → Statistik (**Bootstrap-CI + Inferenz**)
- Zweitgutachten: **Prof. Färber zugesagt** (Exposé, Gespräch Juli)

Sprecher: Die Messinfrastruktur steht und läuft stabil; die offenen Punkte betreffen nur Doku und Reporting, nicht die Daten. Nach dem Stop folgt die Statistik (Bootstrap-CI und Inferenz); Prof. Färber hat das Zweitgutachten zugesagt.

15. Klassifikator & Per-IP-Belege

- Edge nur, wenn **alle drei** zutreffen
- (a) **einstellige ms** FRA :443
- (b) IP in **AS13335** (Team-Cymru)
- (c) Traceroute endet im CDN-AS
- Belegt über **alle getroffenen IPs**
- Deepgram: **zwei US-Carrier** (AS6461 / AS174), ~101–140 ms
- Rev.ai **AS16509** ~140 ms · Azure **AS8075** ~11 ms
- Bei Edge ist connect $\approx N \times \text{RTT}$ zwangsläufig → **kein eigener Beleg**

Sprecher: Der Klassifikator ist über alle getroffenen IPs belegt; bei Edge-Anbietern ist connect $\approx N \times \text{RTT}$ zwangsläufig und zählt nicht — nur host-terminierte Anbieter stützen die Cross-Layer-Brücke. Die ASN-Belege liegen pro IP vor.

16. Aggregation & Inferenz (Detail)

- p50/p90 slot-aufgelöst · p95 nur gepoolt + CI · p99 nicht
- Faustregel: $n \cdot (1-q) \geq 5-10$
- **Stark rechtsschief** → Bootstrap statt t
- E2E: **Monte-Carlo-Faltung** der 3 Phasen
- Joint-Completion / **Pareto-Front** (Latenz vs. Zuverlässigkeit)
- IP-Quelle: **aufgelöste Connect-IP** — Fehlschläge nie über Null-Werte filtern

Sprecher: Wegen der starken Rechtsschiefe nutze ich Bootstrap statt t-Verteilung, p99 ist aus $n=100$ je Slot nicht belastbar, und E2E entsteht per Monte-Carlo-Faltung der drei Phasen.

17. Fehlerbehandlung & Verfügbarkeit

- Erfolg = **inhaltlich gültiger Output** (nicht nur Verbindung)
- Timeout-Filter: Teilstring „**ReadTimeout**“ klassifizieren
- nicht nach `error=='timeout'` (verfehlt fast alle echten Timeouts)
- OpenAI-TTS ~**3,1 % ReadTimeouts** (173/5600, Vollkampagne)
- → Latenz nur erfolgreiche Calls + Asterisk

Sprecher: Timeouts werden per Teilstring 'ReadTimeout' klassifiziert, weil ein naiver `error=='timeout'`-Filter die echten Timeouts verfehlt; OpenAI-TTS hat in der Vollkampagne ~3,1 % Fail-Rate (173/5600).

18. Mess-Stack (Code)

- Pro Schicht ein eigenes Werkzeug: **CLI** für L1/L2, **Python-Timer** für die API-Latenz
- STT ohne SDK (**rohe WebSockets**) · IPv4 erzwungen · Cold-Start je Call

Sprecher: Layer 1/2 nutzen Standard-CLI-Werkzeuge (socket, dig, traceroute, tcpdump, openssl); Layer 3 sind eigene Python-Wrapper — httpx für LLM/TTS, rohe websockets für STT, ein run.py-Runner orchestriert die Slots.

19. Endpunkte & Auflösung

- **9 Endpunkte** über 6 Anbieter — live per DNS aufgelöst, per ASN + RTT klassifiziert

Sprecher: Diese Tabelle ist die Kurzfassung; das separate Referenz-Blatt enthält die vollen URLs, Auth-Methoden, alle aufgelösten IPs und die fertigen ping/dig/traceroute-Befehle zum Selbst-Nachmessen.

20. Anatomie einer Latenzzahl

- **ttft** = ab Request-Absenden bis erstes Token — **connect-inklusiv**
- darin: DNS + TCP + TLS = **connect- Σ ~7 ms** (für alle 3 ~gleich)
- davon nur **~1 ms reine TCP-RTT** (= Netznähe, Layer 1) · TLS ~5 ms
- Rest = **Backend** ~60 → ~479 ms → trägt die **~7,3×-Spreizung**

Sprecher: ttft enthält den Verbindungsaufbau; der ist mit ~7 ms für alle drei praktisch gleich (davon ~1 ms TCP-RTT, ~5 ms TLS). Die Spreizung von 67 auf 487 ms lebt also fast vollständig im Backend — das ist der direkte Beleg, dass nicht das Netz die Differenz macht.